Operating-system structures exercises

State the model, intermediate state, and reasoning before checking the separate solutions.

OS02-E01 API and system call

A library function send_record() may copy data to a kernel buffer and enqueue device work.

1. Which steps necessarily execute in kernel mode?
2. Does library success imply durable data?
3. What must the kernel validate when copying a user pointer?

OS02-E02 Microkernel trade-offs

A claim says moving every filesystem/network service outside the kernel makes the system safer.

1. Explain the supporting isolation argument.
2. Identify at least two costs or new risks.
3. Give a mitigation that does not assume architecture alone establishes safety.

OS02-E03 Boot trust chain

Firmware loads an unsigned loader, which loads a signed kernel.

1. Can the kernel signature stop this loader replacing the kernel?
2. Where must verification occur for a verifiable chain?
3. What controls must accompany module signatures?

OS02-E04 Short writes and copy invariant

Fixed source 6000 B, empty destination, buffer 4096 B, no other writer. Reads return 4096,1904,0; positive writes return 1000,3096,1024,880.

1. Trace both offsets and total read/write bytes.
2. A faulty variant performs only one write per read. Its first writes for the two chunks return 1000 and1024. How many bytes are lost? Do not reuse3096, which belonged to a different remaining-range request, as a return from the new1904 B request.
3. State a general invariant and handle short writes/EINTR.
4. Does destination fsync alone establish a durable new name and atomic replacement?

OS02-E05 Pipeline references and EOF

Pipe gives fd3 read/fd4 write. Shell forks cat and wc. Cat dup2s4 to stdout; wc dup2s3 to stdin.

1. List post-exec pipe references each process should retain.
2. Cat writes6000 B and exits. Give a concrete reason wc still cannot observe EOF.
3. Can cat fail opening nonexistent a while wc outputs0 and exits0?
4. Do two forks and two execs create four children?

OS02-E06 ABI range and TOCTOU

A model ABI copies24 B arguments and an8 B pathname. size_t is32-bit, user upper bound U=0xC0000000, ranges are nonempty, and elements occupy4 B.

1. Compute copied bytes and distinguish API/ABI/implementation.
2. For p=0xBFFFFF00, test byte lengths0x100 and0x101. Give an overflow-safe condition.
3. Compute mathematical and truncated values of0x40000001×4.
4. Why is checking a VMA before direct dereference insufficient?

OS02-E07 Policy and layer dependency

One CPU, A/B/C bursts5/3/1 ms, all ready0, RR order A, B, C, no I/O/switch cost. Separately, pager→driver→scheduler→pager.

1. Compute completion/response for q 1 and q 4.
2. Compute mean response and actual task-switch counts.
3. Identify mechanisms and policy.
4. Why is the cycle incompatible with strict layers, and how can it be removed concretely?

OS02-E08 Trace, tail latency, and race

Request starts0, blocks1, ready8, runs9, returns 10. Two threads separately increment x=0 non-atomically. Latencies are1,1,1,1,1,1,1,100 ms.

1. Split10 ms into CPU, blocked, and ready wait.
2. Compute mean and nearest-rank p99.
3. Give a lost update and explain printf's possible masking effect.
4. Does a mutex around only each write repair the race?

OS02-E09 Root closure and trust chain

Root uses storage S/filesystem F. Other requirements are built in. Modules are accessible only from initramfs or mounted root. A separate firmware→loader→kernel signature policy applies.

1. Can root mount if S is root-only and F built-in?
2. Why do S/F in initramfs still not prove successful boot?
3. Is a kernel signature checked solely by an untrusted loader sufficient?
4. Distinguish digest measurement, valid signature, and anti-rollback.

OS02-E10 Polling, interrupts, and DMA

A device finishes at t=4.2 ms. Polls start at 0.5, 1.0, … ms, each using 0.01 ms of CPU; starts are exact and status is immediately visible. In another scheme an interrupt is delivered immediately and its handler uses 0.04 ms of CPU.

1. Find the first detecting poll, number of polls, detection delay, and polling CPU time.
2. Find the interrupt-handler completion time and CPU service time. If instead the core busy-polls throughout 0–4.2 ms, how much CPU time is used?
3. How can DMA be combined with notification methods? Why is reuse unsafe before buffer ownership is handed back?

OS02-E11 Completion, ready, and running

Continue the interrupt scenario. The handler makes P ready when it finishes, and P first runs at t=4.34 ms. Compute P's ready-queue wait. Explain why device completion, handler completion, P running, and system-call return are distinct events. If the interrupt arrives while the CPU is already in kernel mode, must there be a user-to-kernel transition?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapters 1–2. write(2); fsync(2); pipe(7); Linux 3.2 init/main.c.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study